TTK_ThirdLibs 操作说明

本仓库用于**以 git submodule 方式托管 GitHub 上的第三方开源库**。目前纳入了 **Boost 1.89.0**，并且只初始化了 `geometry` + `serialization` 两个库及其**传递依赖闭包**（共 69 个子模块，库依赖 43 个）。

> 设计原则：Boost 超级项目（`boostorg/boost`）本身有 **170 个嵌套子模块**，一次性全拉很大。这里采用"按需初始化"——只把真正用到的库拉下来，其余保持空（未初始化），需要时再补。

目录结构

```
TTK_ThirdLibs/
├── .gitmodules            # 记录子模块路径 -> 仓库 URL
├── third_party/
│   └── boost/             # Boost 1.89.0 超级项目（submodule，gitlink 固定到
ef7fea3471）
│       ├── .gitmodules    # Boost 内部 170 个子模块的定义
│       ├── libs/...       # 各 Boost 库（按需初始化）
│       └── tools/...      # Boost 构建工具（build/cmake/boost_install/boostdep
等）
└── 操作说明.md            # 本文件
```

`.gitmodules`（本仓库根）：

```
[submodule "third_party/boost"]
    path = third_party/boost
    url = https://github.com/boostorg/boost
```

---

0. 版本是如何固定的（核心机制）

本仓库**没有任何文件写着 "1.89.0" 这样的版本号**——Boost 的版本是靠 git 的 **gitlink**（子模块固定指针）钉死的。

git 索引里，`third_party/boost` 是一个 mode 为 `160000` 的条目，它的"内容"就是 boost 超级项目的一个 commit SHA：

```
$ git ls-tree HEAD third_party/boost
160000 commit ef7fea34711a189472893b88205b1dd3c275677b  third_party/boost
```

`git submodule update --init` 的行为正是**"检出这个 SHA"**，而不是"检出某个 tag"。所以版本的事实来源就是这个 SHA，**换版本 = 换这个 SHA**。

0.1 为什么是确定性的（不会随上游漂移）

根 `.gitmodules` 里只有 `path` 和 `url`，**没有 `branch` 字段**：

```
[submodule "third_party/boost"]
    path = third_party/boost
    url = https://github.com/boostorg/boost
```

没有 `branch` 意味着 git 不会去跟踪上游任何分支的 HEAD；`git submodule update` 只认 gitlink 里的 SHA。若当初写了 `branch = master`，`git submodule update --remote` 就会跟着上游跑，那种配置是不可复现的——当前不是这种。

0.2 验证这个 SHA 就是 Boost 1.89.0

`ef7fea3471` 与 boost 官方 tag `boost-1.89.0` 指向同一个 commit：

```
# 联网校验（GitHub 不稳时可能超时，重试即可）
git ls-remote https://github.com/boostorg/boost refs/tags/boost-1.89.0
# 期望输出：
# ef7fea34711a189472893b88205b1dd3c275677b  refs/tags/boost-1.89.0
```

输出与上面的 gitlink SHA 一致，即说明本仓库确实固定在 1.89.0 上。（`boost-1.89.0` 是轻量 tag，`ls-remote` 直接返回 commit SHA，不会出现 tag 对象的二次跳转。）

0.3 版本是递归继承的

Boost 是被**当作一个整体**钉住的：`ef7fea3471` 这个 commit 的树里，那 170 个嵌套子模块（`libs/*`、`tools/*`）各自也有自己的 gitlink。所以只要 boost 超级项目的 SHA 定了，`libs/geometry`、`libs/serialization` 等拉到的就必然是 1.89.0 配套的版本——**不需要在本仓库里逐个指定**，这也解释了为什么根目录只有一条 submodule 记录。

0.4 升级 Boost 版本

本质就是把 gitlink 换成新 tag 对应的 SHA（通用套路见 §7）：

```
cd third_party/boost
git fetch --tags
git checkout boost-1.90.0        # 换成目标 tag
cd ../..
git add third_party/boost
git commit -m "Bump Boost to 1.90.0"
```

注意两点：

- `.gitmodules` **不用改**（它不含版本信息），变更全部体现在 gitlink 上；
- 换 SHA 后，各子库的 commit 也随之变化，需重新按 §1.2 / §2 初始化对应子模块。

---

1. 克隆本仓库并获取 Boost

1.1 普通克隆（只拿 Boost 超级项目，不拉任何 Boost 库）

```
git clone git@github.com:ToraSpencer/TTK_ThirdLibs.git
cd TTK_ThirdLibs
git submodule update --init third_party/boost
```

此时 `third_party/boost` 是 Boost 1.89.0 的**空骨架**（libs 下只有空目录，没有代码）。

1.2 把已初始化的 Boost 库一起拉下来（推荐）

本仓库当前已初始化了 69 个 Boost 子模块（见文末清单）。要复现同样的状态，进入 boost 目录后初始化对应的子模块：

```
cd third_party/boost
# 一次性把 geometry + serialization 的依赖闭包拉下来（推荐方式，见下节）
python tools/boostdep/depinst/depinst.py -g -- "--force" geometry
python tools/boostdep/depinst/depinst.py -g -- "--force" serialization
```

> 上面的 `depinst.py` 会自动算出依赖闭包并 `git submodule update --init` 需要的库。 注意必须加 `-g -- "--force"`：否则在部分 git 版本上对"已匹配"的子模块会误报 `Unable to find current revision`（见 §6 已知问题）。

如果只想**手动**拉某个库（不经 depinst）：

```
cd third_party/boost
git submodule update --init --force libs/geometry
```

---

2. 用 depinst.py 拉取某个 Boost 库的完整依赖（推荐做法）

Boost 自带依赖安装器 `tools/boostdep/depinst/depinst.py`。它扫描目标库 `include/` 与 `src/` 里的 `#include <boost/...>`，自动推导**头文件依赖闭包**并初始化对应子模块。

```
cd third_party/boost
# 先确保 tools/boostdep 本身已初始化
git submodule update --init --force tools/boostdep

# 拉取某库及其全部依赖闭包（geometry 例）
python tools/boostdep/depinst/depinst.py -g -- "--force" geometry
```

要点：

- `-g -- "--force"`：把 `--force` 传给 `git submodule update`，避免已知误报（§6）。
- 依赖闭包只含**运行时/编译**所需（扫描 `include`+`src`），不含仅测试用库。
- 依赖闭包是**逐层收敛**的：装上一层才能扫描出下一层，所以第一次运行可能只装一部分，重复运行会继续补齐（正常情况下自动到不动点）。

---

3. 校验依赖是否完整（离线，不联网）

在 boost 目录里，用下面的脚本判断某个库的依赖闭包是否已全部初始化（**不联网**）：

```
cd third_party/boost
python -c "
import sys, os
os.chdir('.')
sys.path.insert(0,'tools/boostdep/depinst');
sys.argv[0]='tools/boostdep/depinst/depinst.py'
import depinst
gm=depinst.read_gitmodules(); x=depinst.read_exceptions()
installed={l.split()[1][5:] for l in os.popen('git submodule
status').read().splitlines() if l[:1]==' ' and l.split()[1].startswith('libs/')}
deps={}
for tgt in ['geometry','serialization']:   # <== 改成你要查的库
    depinst.scan_module_dependencies(tgt,x,gm,deps,['include','src'])
for m in list(deps):
    if deps[m]: depinst.scan_module_dependencies(m,x,gm,deps,['include','src'])
for m in list(deps):
    if deps[m]: depinst.scan_module_dependencies(m,x,gm,deps,['include','src'])
missing={m for m,s in deps.items() if not s and m not in installed}
print('依赖闭包库数:', len(deps))
print('缺失:', sorted(missing) if missing else '（无，完整）')
"
```

---

4. 查看当前状态

```
# 本仓库（超级项目）状态
git status

# 查看 Boost 被固定在哪个 commit（即"版本"本身，见 §0）
git ls-tree HEAD third_party/boost
git submodule status third_party/boost      # SHA 前无 +/- 前缀 = 本地检出的就是它

# Boost 内部：哪些子模块已初始化（空格前缀=已就绪；- 前缀=未初始化；+ 前缀=commit 不符）
cd third_party/boost
git submodule status

# 只列出已就绪的
git submodule status | grep '^ '
```

```
# 统计已就绪数量
git submodule status | grep -c '^ '
```

---

5. 网络不稳定（到 GitHub 间歇性超时 / connection reset）

本仓库所处环境访问 GitHub **不稳定**，拉子模块经常失败。应对手段：

5.1 调低 git 超时，避免长时间挂起

在 boost 目录（或本仓库根）执行一次：

```
git config http.connectTimeout 30     # 连接 30s 上限
git config http.lowSpeedLimit 1000    # 1 KB/s
git config http.lowSpeedTime 60       # 低速持续 60s 即判失败
git config submodule.fetchJobs 4      # 并发拉取 4 个
```

> 注意：`git config`（无 `--global`）写入的是**当前仓库**的本地配置。在 `third_party/boost` 里执行，就写进 boost 子模块。

5.2 单个库带重试拉取

对某个库反复重试，直到成功（脚本会自退避）：

```
cd third_party/boost
LIB=libs/variant
for i in $(seq 1 60); do
  if git submodule status -- "$LIB" | grep -q '^ '; then echo "OK $LIB"; break; fi
  git submodule deinit -f "$LIB" 2>/dev/null; rm -rf "$LIB"
  git submodule update --init --force "$LIB" && break
  echo "retry $i for $LIB"; sleep 10
done
```

5.3 判断网络是否已恢复

```
git ls-remote https://github.com/boostorg/headers.git HEAD
# 能返回一个 commit SHA 即网络通；超时/失败即暂不可用
```

---

6. 已知问题 / 坑

1. **`git submodule update`（不带 `--force`）误报** 在 git 2.45 上，对一个"已匹配固定 commit"的子模块执行非 `--force` 的 `git submodule update`，可能报 `fatal: Unable to find current revision in submodule path '...'`。 这不是真的缺对象，是 quirk。**用 `--force` 即可**（见 §1.2、§2、§5.2）。

2. **depinst.py 的 `-g` 参数传 `--force` 要用 `--` 分隔** 直接 `-g --force` 会被 argparse 当成标志。写成 `-g --"--force"`。
3. **中断留下的半成品子模块** 网络中断时，某子模块目录可能克隆了一半（`.git` 在，但对象不全），之后再拉会报错。 处理方式：`git submodule deinit -f <path> && rm -rf <path>` 后重新 `git submodule update --init --force <path>`。
4. **boost 固定 commit 都在各子库的默认分支 HEAD 上** Boost 1.89.0 记录的各子模块 commit，通常就是对应库 `master` 的最新 commit（如 `libs/headers` HEAD = `95930ca8...`）。所以只要网络通，全量克隆即可取到；失败几乎都是网络截断，重试即可。
5. **序列化库的头文件主入口是 `boost/serialization/serialization.hpp`**（不是 `serialize.hpp`）。

---

7. 在本仓库新增其它第三方库

通用套路（以任意 GitHub 仓库为例）：

```
# 1) 添加 submodule（可选：用 --branch 跟踪分支；默认跟踪分支 HEAD）
git submodule add https://github.com/<owner>/<repo>.git third_party/<repo>

# 2) 固定到具体版本（推荐固定到 tag，保证可复现）
#    这一步产生的就是 §0 所说的 gitlink —— 版本不在 .gitmodules 里，
#    而在索引的这个 160000 条目上
cd third_party/<repo>
git checkout <tag-or-commit>
cd ../..

# 2.5) 确认固定结果（应看到一个 160000 commit 条目）
git ls-tree HEAD third_party/<repo>

# 3) 提交本仓库（记录新的 gitlink 与 .gitmodules）
git add .gitmodules third_party/<repo>
git commit -m "Add <repo> as submodule"
```

若某个库**本身又有子模块**（像 Boost），克隆方需要：

```
git submodule update --init third_party/<repo>      # 拿到该库
cd third_party/<repo>
git submodule update --init <需要的子路径>            # 只拉需要的子模块
```

---

8. 推荐 git 配置（一次性）

在本仓库根执行：

```
git config http.connectTimeout 30
git config http.lowSpeedLimit 1000
git config http.lowSpeedTime 60
git config submodule.fetchJobs 4
```

附：当前已初始化的 Boost 子模块（69 个）

构建基础：

- libs/config、libs/headers、tools/boost_install、tools/build、tools/cmake、tools/boostdep

目标库：

- libs/geometry、libs/serialization

geometry + serialization 的传递依赖（库）：

- libs/algorithm、libs/any、libs/array、libs/assert、libs/bimap、libs/bind、libs/chrono、libs/concept_check、libs/container、libs/container_hash、libs/conversion、libs/core、libs/detail、libs/exception、libs/foreach、libs/function、libs/function_types、libs/functional、libs/fusion、libs/graph、libs/integer、libs/intrusive、libs/io、libs/iterator、libs/lexical_cast、libs/math、libs/move、libs/mp11、libs/mpl、libs/multi_index、libs/multiprecision、libs/numeric/conversion、libs/optional、libs/parameter、libs/polygon、libs/predef、libs/preprocessor、libs/property_map、libs/property_tree、libs/qvm、libs/random、libs/range、libs/rational、libs/regex、libs/smart_ptr、libs/spirit、libs/static_assert、libs/system、libs/thread、libs/throw_exception、libs/tokenizer、libs/tti、libs/tuple、libs/type_index、libs/type_traits、libs/typeof、libs/unordered、libs/utility、libs/variant、libs/variant2、libs/xpressive

> 说明：其余 Boost 库（约 100 个）未初始化。需要时按 §1.2 / §2 / §5.2 补拉即可。